Soft Nexis — Data Science Internship

Task 1 Documentation: Data Cleaning & Preprocessing Pipeline

TASK 1 COMPLETION

1. Executive Summary & Objective

Raw datasets in real-world applications frequently contain noise, missing fields, duplicate entries, and inconsistent formatting. This report outlines the standardized 6-step workflow implemented for Task 1, transforming unstructured operational records into an analysis-ready tabular dataset.

2. Six-Step Cleaning Methodology

Step 1: Data Ingestion

Load raw datasets via `pd.read_csv()` or SQL connections. Perform primary sanity checks on dataset shape, features, and baseline info.

Step 2: Deduplication

Detect exact row duplicates and cross-column redundant records using `df.duplicated()`. Drop duplicates while retaining first observations.

Step 3: Column Management

Eliminate uninformative columns (e.g., internal IDs, temporary notes). Standardize headers to `snake_case` naming convention.

Step 4: Missing Value Handling

Diagnose MCAR, MAR, or MNAR patterns. Apply column deletion if >70% missing; apply median/mode imputation or interpolation for remaining gaps.

Step 5: Data Type Correction

Cast raw text strings into `datetime64` objects. Strip currency/punctuation symbols from string numbers and convert to numeric floats.

Step 6: Format Standardization

Normalize text via lowercase and whitespace trimming. Standardize categorical variations (e.g., mapping "M" / "Male" to unified categories).

3. Pipeline Verification Matrix

Pipeline Stage	Raw Condition	Applied Method	Target Outcome
Duplicates	Redundant rows & IDs	`df.drop_duplicates()`	0 duplicate rows
Column Names	Mixed case & spaces	Regex / `str.lower()`	Clean `snake_case`
Missing Values	Nulls across features	Median / Mode Imputation	Complete dataset
Data Types	Dates & numbers as string	`pd.to_datetime()`	Numeric & datetime objects
Text Encodings	Leading spaces & synonyms	`str.strip()` & map	Unified string fields

Validation Guardrail: Before exporting processed data, explicit assertion guards (e.g., `assert df.duplicated().sum() == 0`) execute to prevent silent failure propagation to downstream analytics.

4. Production Pipeline Code (`data_cleaning_pipeline.py`)

The following modular Python script automates the complete data cleaning workflow as required for Task 1 execution:

```
"""
Soft Nexis Internship - Task 1
Data Cleaning & Preprocessing Pipeline
"""

import pandas as pd
import numpy as np


class DataCleaner:
    def __init__(self, file_path: str):
        self.file_path = file_path
        self.df = None

    def ingest_data() -> pd.DataFrame:
        """Step 1: Load raw dataset and inspect dimensions."""
        self.df = pd.read_csv(self.file_path)
        print(f"[+] Loaded raw dataset: {self.df.shape[0]} rows, {self.df.shape[1]} cols")
        return self.df

    def remove_duplicates(self, subset_cols: list = None) -> pd.DataFrame:
        """Step 2: Detect and drop redundant rows."""
        initial_rows = len(self.df)
        self.df = self.df.drop_duplicates(subset=subset_cols, keep="first")
        print(f"[+] Deduplication complete: Removed {initial_rows - len(self.df)} rows.")
        return self.df

    def manage_columns(self, drop_cols: list = None) -> pd.DataFrame:
        """Step 3: Drop unneeded features & standardize headers to snake_case."""
        if drop_cols:
            self.df = self.df.drop(columns=drop_cols, errors="ignore")

        self.df.columns = (
            self.df.columns.str.strip()
            .str.lower()
            .str.replace(" ", "_")
            .str.replace("[^a-zA-Z0-9_]", "", regex=True)
        )
        print("[+] Headers converted to standard snake_case.")
        return self.df

    def handle_missing_values(self, missing_thresh: float = 0.70) -> pd.DataFrame:
        """Step 4: Drop empty columns and impute missing data."""
        threshold = int((1 - missing_thresh) * len(self.df))
        self.df = self.df.dropna(thresh=threshold, axis=1)

        for col in self.df.columns:
            if self.df[col].isna().sum() > 0:
                if np.issubdtype(self.df[col].dtype, np.number):
                    self.df[col].fillna(self.df[col].median(), inplace=True)
                else:
                    mode_val = self.df[col].mode()[0] if not self.df[col].mode().empty else "Unknow
n"
                    self.df[col].fillna(mode_val, inplace=True)
        print("[+] Missing value imputation complete.")
        return self.df

    def correct_data_types(self, date_cols: list = None, numeric_cols: list = None) -> pd.DataFram
```

5. Repository Documentation (`README.md`)

Below is the markdown file structure configured for direct inclusion in the GitHub submission repository:

````
# Data Cleaning & Preprocessing Pipeline

![Python](https://img.shields.io/badge/Python-3.8%2B-blue)
![Pandas](https://img.shields.io/badge/Pandas-Latest-orange)
![Build](https://img.shields.io/badge/Status-Passing-brightgreen)

## Executive Summary
This repository contains an end-to-end data cleaning pipeline developed during my data science inte
rnship at Soft Nexis. The pipeline accepts raw CSV inputs and executes automated deduplication, col
umn normalization, missing value imputation, and type conversion.

---

## Repository Structure

```text
.
├── data/
│   ├── raw/                  # Source CSV datasets
│   └── processed/            # Cleaned output CSVs
├── data_cleaning_pipeline.py # Core Python transformation module
├── README.md                 # Project documentation
└── requirements.txt          # Dependencies
```

---

## Execution Instructions

1. **Clone Repository:**
   ```bash
   git clone https://github.com/your-username/data-cleaning-pipeline.git
   cd data-cleaning-pipeline
   ```

2. **Install Dependencies:**
   ```bash
   pip install pandas numpy
   ```

3. **Run Cleaning Pipeline:**
   ```bash
   python data_cleaning_pipeline.py
   ```

---

## Pipeline Outcomes

| Metric | Raw Dataset | Processed Output |
| :--- | :--- | :--- |
| **Duplicates** | Present | 0 (Enforced) |
| **Header Format** | Mixed Case / Spaces | Standardized `snake_case` |
| **Missing Values** | Unhandled Gaps | Imputed (Median / Mode) |
| **Date Fields** | Plain Text Strings | Native `datetime64` |
````

6. Git Submission Commands